FREE AI CODING IN YOUR TERMINAL

A complete beginner's guide to FCC + Claude Code + NVIDIA NIM

Start from zero • Follow the steps in order • Copy the commands • Fix common errors

Who this guide is for

This guide is written for a person who has never configured an AI coding model in a terminal. It follows the setup you used: Windows PowerShell, FCC, Claude Code, and NVIDIA NIM using a Nemotron model. You already got a successful response and built a website; this document explains the process from the beginning so another beginner can follow it.

Important: this is a guide to routing Claude Code to a free-to-use NVIDIA model. It is not the same as using Anthropic's own Claude model for free. The exact free availability and limits depend on NVIDIA's current account and model rules.

Read the PDF in order. Do not skip to troubleshooting unless you have already completed the setup steps.

Contents

1. Understand the tools
2. What to prepare
3. Install/check prerequisites
4. Open FCC Admin
5. Configure NVIDIA NIM
6. Connect and select the model in Claude Code
7. Test the connection
8. Start a project safely
9. Copy-ready prompts for terminal work
10. Error-fixing table
11. Understand free limits
12. Final checklist

1. Understand what each tool does

PowerShell — the Windows terminal where you type commands.
Claude Code — the coding assistant interface you launch by typing `claude`.
FCC — the local configuration/routing layer used in your setup to connect the terminal assistant to a chosen model provider.
NVIDIA NIM — the hosted model service providing the Nemotron model response.
API key — a secret password-like token that authorizes requests to the provider.

How your setup works

```
You type in PowerShell
        ↓
Claude Code terminal
        ↓
FCC routing/configuration
        ↓
NVIDIA NIM API + selected Nemotron model
        ↓
The model response returns to the terminal
```

2. Before you begin

You need a Windows PC with internet access, PowerShell, Node.js/npm if required by your FCC installation, Claude Code, FCC, and an NVIDIA account/API key with access to a supported model. The earlier setup had FCC Admin available at:

```
http://127.0.0.1:8082/admin
```

If that page does not open, FCC may not be running or may use a different port. Find the terminal/window used to start FCC and read its output; do not assume every installation uses the same address.

3. Check your prerequisites

Open PowerShell from the Windows Start menu. Run these commands one at a time:

```
node -v
npm -v
claude
```

What success looks like: Node and npm print version numbers; `claude` starts Claude Code. If a command says “not recognized,” see the error-fix section before continuing.

Do not install every tool again if it already works. Your previous setup had FCC and Claude Code installed successfully, so a new reader should first check the commands and installed apps.

4. Open FCC Admin

1. Start FCC using the same method used during installation.
2. Open your browser and visit `http://127.0.0.1:8082/admin`.
3. Find the provider/API key settings and model configuration section.
4. If the page is unavailable, check the FCC terminal for startup errors and the actual local address.

Your earlier FCC setup showed 27 available models and used the NVIDIA NIM provider. The number and list of models may change over time.

5. Get and add an NVIDIA NIM API key

1. Sign in to NVIDIA's official developer/model catalog: https://build.nvidia.com/.
2. Choose a model that is currently available to your account and follow the provider's current instructions for creating or accessing an API key.
3. Copy the key privately.
4. Return to FCC Admin and paste it into the NVIDIA NIM provider/API-key field.
5. Save the settings.
6. Confirm the model is listed and can be selected.

Never share your API key in a screenshot, public post, source-code repository, or chat. If exposed, revoke it and create a replacement.

6. Choose the model in FCC

In your earlier working setup, the selected model identifier was:

```
nvidia_nim/nvidia/nemotron-3-ultra-550b-a55b
```

This is a model identifier from your previous setup, not a promise that it will remain available. If it is missing or unavailable, select a currently listed NVIDIA NIM model that your account can access. Set the default in FCC only if you want new sessions to use it by default.

If FCC has a field labelled “usage limit” or “unlimited,” understand it as a local setting unless the interface explicitly says it controls provider quota. It cannot remove NVIDIA's own limits.

7. Start Claude Code and select the NVIDIA model

Open a fresh PowerShell window. Go to a folder you can use for testing and launch Claude Code:

```
cd C:\Users\karth
claude
```

Inside the Claude Code interactive terminal, type:

```
/model
```

Choose the NVIDIA NIM Nemotron model if it appears. In your earlier troubleshooting, `%USERPROFILE%\.claude\settings.json` had a model value pointing to a Claude-named model. You changed the value to `default`, then selected the NVIDIA NIM model using `/model`.

If you edit the settings file, make a backup first. Do not replace the entire JSON file with a sample from the internet; it may contain other settings. Prefer the in-app `/model` selector when it works.

8. Test that the connection works

Inside Claude Code, send this small test prompt:

```
Reply with exactly: NEMOTRON TEST PASSED
```

Your earlier setup returned “NEMOTRON TEST PASSED.” That confirmed the model could respond. If you receive a different response, a model-selection or routing issue may remain; inspect the actual output.

9. Start a project without mixing files

When the model test works, create a separate folder for each project. Example:

```
mkdir C:\Users\karth\my-project
cd C:\Users\karth\my-project
claude
```

If the folder already exists, just use `cd` to open it. Keep each project in its own folder. Do not run cleanup commands from a folder unless you understand what they affect.

To clear the current conversation context for an unrelated task, you can use `/clear` inside Claude Code. It clears conversation context; it does not delete project files.

10. Copy-ready prompts for safer terminal work

These prompts are for controlling and troubleshooting your coding assistant in the terminal—not for building one particular website.

Prompt A — inspect before acting

```
Before making changes, inspect the current folder and explain:
1. What tools and files are present?
2. What step should I do next?
3. Which files or settings would be changed?
Do not delete, overwrite, or install anything without explaining why.
Ask me before any destructive action.
```

Prompt B — stay inside the current project

```
Work only inside this folder:
C:\Users\karth\my-project

Inspect existing files first. Preserve working code. Do not modify or delete
files outside this folder. If a command may delete data or change system
settings, explain it and ask me before proceeding.
```

Prompt C — diagnose a terminal error

```
Diagnose this exact error:
[PASTE THE COMPLETE ERROR HERE]

Explain the likely cause in beginner-friendly language.
Check relevant settings, versions, and files before changing anything.
Make the smallest safe fix, rerun the failed command, and report the result.
Do not claim success unless the command output confirms it.
```

Prompt D — continue after a session stops

```
Continue from the current project state. First inspect what exists,
what is complete, and what is missing. Do not recreate working files.
Finish only the missing tasks, run the relevant verification command,
fix errors, and report the actual result. Do not claim completion unless
verification succeeds.
```

Prompt E — explain commands before I approve them

```
I am a beginner. Before suggesting a command, explain what it does,
which folder or settings it affects, and whether it can delete or overwrite
anything. Prefer safe, reversible steps and ask before destructive actions.
```

How to use a prompt

Copy the whole prompt, paste it into the Claude Code terminal, replace the bracketed text with the actual error or folder, and press Enter. Review any command or file-change permission request before accepting it.

11. Common errors and fixes

Error / symptom	Beginner-friendly fix
FCC preflight or OpenCode conflict	In your earlier setup, uninstalling a conflicting old global package resolved the preflight issue. Only if the same conflict is shown, run `npm uninstall -g opencode-ai` and retry the relevant setup step. Do not run this command unless the conflict matches.
FCC Admin page will not open	Check that FCC is running. Read the terminal output for its actual local URL/port. The earlier setup used `http://127.0.0.1:8082/admin`.
Wrong model is selected	Inside Claude Code, run `/model` and select the intended NVIDIA NIM model. Check FCC's provider and model configuration.
Model does not appear in list	Check the provider/key configuration and whether that model is currently available to your account. Select a currently listed supported model instead of assuming an old model ID still works.
429 / rate limit exceeded	Pause and retry later. Check the current provider/model limits. A local FCC setting labelled unlimited does not override NVIDIA-side rate limits.
Quota / access denied / authentication error	Check the key was entered in the correct provider field, that it is valid, and that your account has access to the model. Never paste the key into the prompt.
claude is not recognized	Confirm Claude Code is installed and its executable is on PATH. Reopen PowerShell after installation. If still failing, check the installation instructions used for your version.
node or npm is not recognized	Install a supported Node.js LTS release if missing, reopen PowerShell, then run `node -v` and `npm -v`.
Settings JSON error	Back up `%USERPROFILE%\.claude\settings.json`. Check for missing commas/quotes and edit only the intended setting. Avoid replacing the whole file with guessed JSON.
Model answers but a tool/action fails	Text response only proves that the model answered. Check the separate tool permission, folder access, and exact tool error. Approve only actions you understand.

Universal error prompt

```
I encountered this error:
[PASTE ERROR, but remove API keys and private data]

Identify which layer failed: PowerShell/installation, FCC, provider authentication,
quota/rate limit, model selection, tool permission, or project code.
Explain the cause simply. Suggest the smallest safe fix and verify it with
the exact command. Ask before destructive changes. Report what actually happened.
```

12. Free usage, limits, and safety

You said you configured FCC's usage limit as “unlimited.” This may remove a local FCC cap, but it does not guarantee unlimited free requests from NVIDIA.

Different limits can apply independently:

- FCC's local usage setting
- NVIDIA's per-model request rate limits
- NVIDIA account/free-access quota and availability
- Model access or service changes
- The model's context window for a long conversation

If you hit a limit, save your work, note the exact error without secrets, check the provider's current account/model information, and wait if it is a temporary rate limit. Do not assume a fixed number of websites or hours before the limit.

Never publish screenshots that show API keys, access tokens, account secrets, or private settings. If a key is exposed, revoke it and create a new one.

13. Final checklist

- ■ PowerShell commands work
- ■ FCC is running and its Admin page opens
- ■ NVIDIA NIM key is saved privately
- ■ A currently available NVIDIA model is selected
- ■ `/model` shows the intended model in Claude Code
- ■ A small test request receives a response
- ■ You know how to create/open a project folder
- ■ You know how to paste the error-fixing prompt
- ■ You keep backups and review risky commands
- ■ You understand that “unlimited” in FCC does not guarantee unlimited provider usage

Reference note

This guide is based on the steps and errors described in your conversation. FCC menus, model names, and NVIDIA access rules can change; if the current interface differs, follow the labels and errors shown on your screen. No unverified quota or guaranteed free usage amount is promised.